

Practical Activity - Implementing the Factory Method Pattern

In this practical activity, I implemented the Factory Method pattern in a car manufacturing system that produces Sedan, SUV and Hatchback objects without directly instantiating the concrete car classes in the main program. The implementation includes an abstract Car class, three concrete car classes, an abstract CarFactory, and three concrete factories.

Python Implementation

1. Abstract Car Class:

The Car class defines the common drive() behaviour that every concrete car type must implement.

```
from abc import ABC, abstractmethod

class Car(ABC):

    @abstractmethod
    def drive(self) -> None:
        """
        Define the common drive behaviour.

        Every concrete car class must implement
        its own version of this method.
        """
        raise NotImplementedError
```

2. Concrete Car Class:

The following classes represent the actual car objects that the factories will create. Each class inherits from Car and provides its own implementation of drive().

```
class Sedan(Car):

    def drive(self) -> None:
        """
        Sedan-specific implementation of the drive method
```

```
    """
    print("Driving a Sedan ")

class SUV(Car):

    def drive(self) -> None:
        """
        SUV-specific implementation of the drive
        """
        print("Driving an SUV ")

class Hatchback(Car):

    def drive(self) -> None:
        """
        Hatchback-specific implementation of the drive
        """
        print("Driving a Hatchback ")
```

3. Factory Classes:

CarFactory defines the common create_car() factory method. Each concrete factory is then responsible for creating a specific type of car.

```
class CarFactory(ABC):

    @abstractmethod
    def create_car(self) -> Car:
        """
        Factory Method.

        Concrete factory classes must implement this
        method and return a Car object.
        """
        raise NotImplementedError

# Each factory is responsible for manufacturing a specific type of car
class SedanFactory(CarFactory):

    def create_car(self) -> Car:
        """
        Create and return a Sedan object.
        """
        return Sedan()

class SUVFactory(CarFactory):

    def create_car(self) -> Car:
        """
        Create and return an SUV object.
        """
        return SUV()

class HatchbackFactory(CarFactory):

    def create_car(self) -> Car:
        """
        Create and return a Hatchback object.
        """
        return Hatchback()
```

4. Using the Factory:

The `demonstrate_factory()` function does not need to know whether the supplied factory creates a Sedan, SUV or Hatchback. It only asks the factory to create a car and then calls its `drive()` method.

```
def demonstrate_factory(factory: CarFactory) -> None:
    car = factory.create_car()
    car.drive()
```

5. The main program

The main program creates the factory objects and passes each factory to `demonstrate_factory()`.

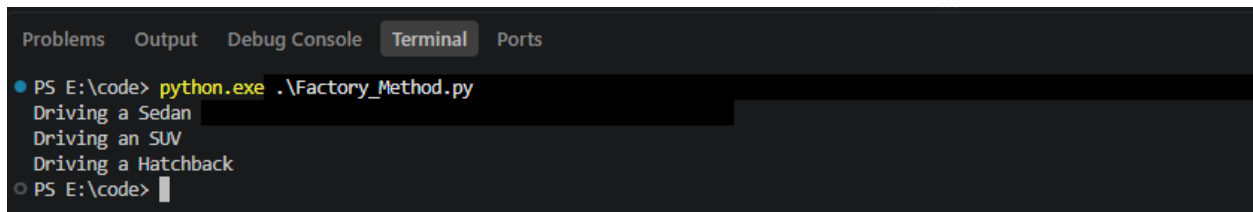
```
if __name__ == "__main__":

    # Each factory knows how to create one specific type of car.
    factories = [
        SedanFactory(),
        SUVFactory(),
        HatchbackFactory()
    ]

    # Go through every factory.
    for factory in factories:

        demonstrate_factory(factory)
```

6. Program Output:

A screenshot of a Visual Studio Code terminal window. The terminal has tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active. The command prompt shows 'PS E:\code> python.exe .\Factory_Method.py'. The output of the script is displayed on the following lines: 'Driving a Sedan', 'Driving an SUV', and 'Driving a Hatchback'. The prompt returns to 'PS E:\code>'.

The program work flow:

- Car is the abstract product and defines the common drive() behaviour expected from every car type.
- Sedan, SUV and Hatchback are concrete products that inherit from Car and provide their own drive() implementation.
- CarFactory is the abstract creator and declares the create_car() factory method.
- SedanFactory, SUVFactory and HatchbackFactory are concrete creators that override create_car() and return the appropriate product.
- The main program works through factory objects and does not directly instantiate Sedan, SUV or Hatchback.